

Gradius ERP — 개발자 기술 가이드

대상: 신규 개발자 및 시스템 유지보수 담당자
최종 수정: 2026년 4월
버전: v1.5 (master 브랜치 기준)

목차

- [1. 프로그램 개요](#)
- [2. 기술 스택 및 의존성](#)
- [3. 시스템 아키텍처](#)
- [4. 디렉토리 구조 및 파일 역할](#)
- [5. 데이터 계층 — Google Sheets 구조](#)
- [6. 업무 파이프라인 \(7단계 상태 흐름\)](#)
- [7. 인증 및 GCP 설정](#)
- [8. 데이터 로더 API 레퍼런스](#)
- [9. 캐싱 전략](#)
- [10. 페이지 모듈 레퍼런스](#)
- [11. 비즈니스 로직 모듈](#)
- [12. AI 및 자동화](#)
- [13. Session State 키 맵](#)
- [14. 개발 환경 설정](#)
- [15. 배포 가이드](#)
- [16. 주요 설계 패턴](#)
- [17. 알려진 기술 부채](#)
- [18. 개발 시 주의사항](#)

1. 프로그램 개요

Gradius ERP는 스태프 파견 및 이벤트 행사 전문업체를 위한 **통합 행정 자동화 웹 애플리케이션**입니다.

핵심 가치

가치	설명
자동화	문의 접수부터 정산 완료까지 수동 작업 최소화
실시간	Google Sheets 기반 공유 데이터, 30분 캐시로 빠른 응답
AI 지원	Gemini AI로 경영 인사이트, 견적 추천, 자연어 조회
한국어 전용	UI/데이터/용어 모두 한국어

사용자 역할

- **대표(CEO):** 전용 대시보드에서 미수금/미지급/수익률 모니터링 및 인사 컨펌
- **영업/기획:** 문의 접수, 견적 작성, 계약 처리
- **운영/배정:** 인력 배정, 출석 관리
- **정산 담당:** 고객 청구, 인력 지급, 세금계산서

2. 기술 스택 및 의존성

핵심 라이브러리

```
streamlit>=1.0           # 웹 프레임워크 (UI 전체)
pandas>=1.5              # 데이터 처리 (시트 데이터 ↔ DataFrame)
gsread>=5.7.0            # Google Sheets API 클라이언트
oauth2client>=4.1.3      # GCP 서비스 계정 인증
google-auth>=2.0.0       # Google 인증 라이브러리
google-api-python-client>=2.0 # Google API (Calendar 등)
google-genai>=1.0        # Gemini AI API
plotly>=5.0              # 차트 및 시각화
apscheduler>=3.10        # 백그라운드 스케줄러
Pillow>=9.0              # 이미지 처리 (견적서 생성)
openpyxl>=3.0            # Excel 호환
streamlit-calendar>=0.1  # 캘린더 위젯
numpy>=1.24              # 수치 계산
requests>=2.28           # HTTP 요청
python-dotenv>=0.20      # .env 파일 지원
pydantic>=1.9            # 데이터 검증
```

시스템 패키지 (packages.txt)

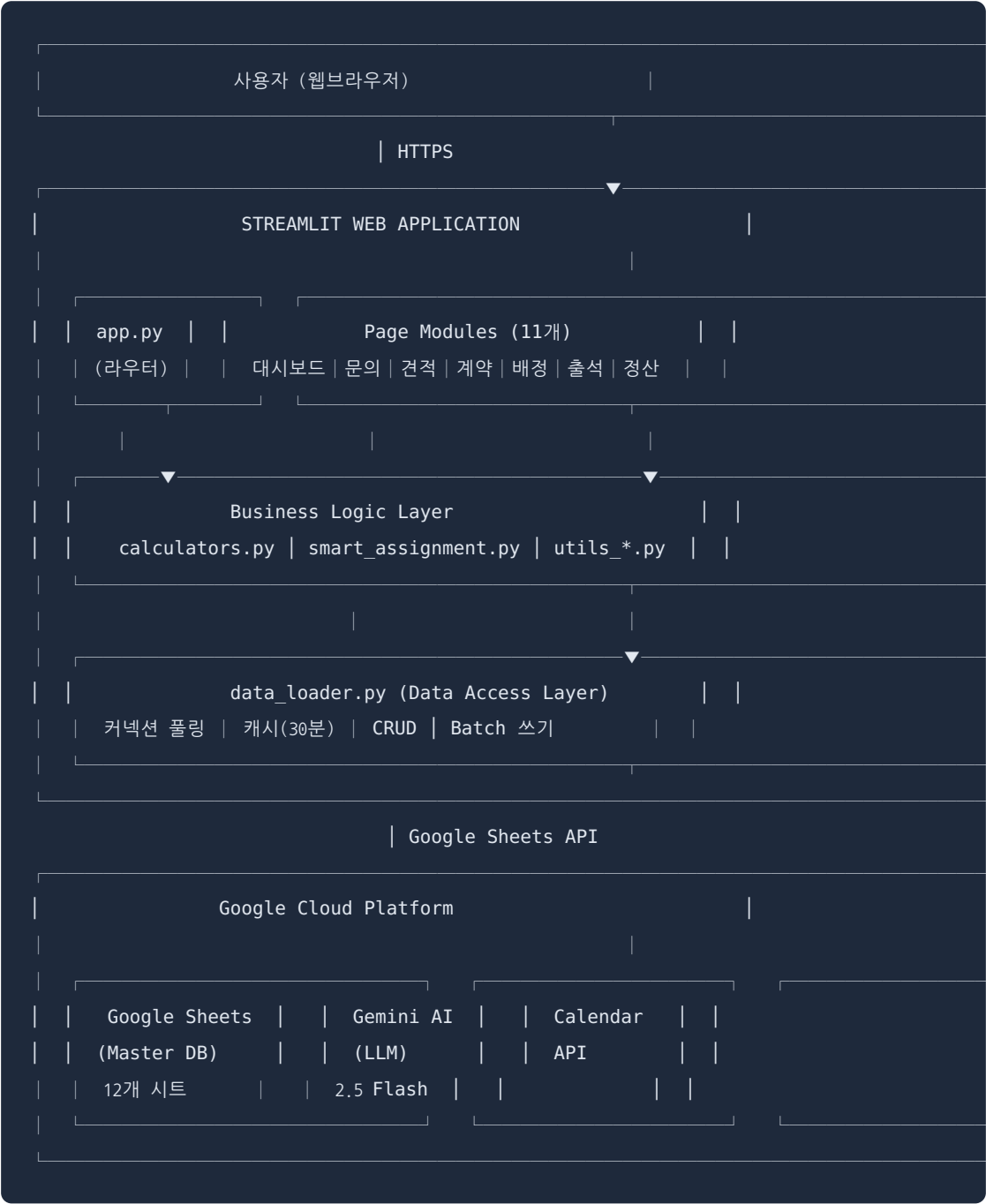
```
fonts-nanum              # 한글 나눔 폰트 (견적서 이미지 생성 필수)
```

Python 버전

Python 3.10 이상 권장 (3.9+ 호환)

3. 시스템 아키텍처

전체 구조



데이터 흐름

사용자 액션

- |
- ▼ (1) UI 이벤트 → session_state 업데이트
- |
- ▼ (2) 비즈니스 로직 처리 (calculators, utils)
- |
- ▼ (3) data_loader 통해 Google Sheets 쓰기
- |
- ▼ (4) 캐시 무효화 (invalidate_*)
- |
- ▼ (5) st.rerun() → 화면 재렌더링
- |
- ▼ (6) get_data() → 세션 캐시 체크 → 없으면 Sheets에서 읽기

4. 디렉토리 구조 및 파일 역할

진입점 및 라우팅

파일	역할
<code>app.py</code>	메인 진입점. 사이드바 메뉴 정의, 페이지 라우팅, 전역 캐시 관리, 데이터 동기화 버튼
<code>auth.py</code>	GCP 서비스 계정 인증. <code>get_gspread_client()</code> 제공

app.py 라우팅 맵

```
_menu_items = [
    "☐ 대표님 전용",          # → page_ceo.show()
    "☐ 경영 대시보드",       # → page_dashboard.show()
    "☐ 문의 접수 및 관리",    # → page_inquiry.show()
    "☐ 견적 통합 관리",       # → page_estimate.show()
    "☐ 계약 관리 및 승인",    # → page_contract.show()
    "☐ 인원 배정 관리",       # → page_staff_new.show()
    "☐ 출석부 관리",          # → page_attendance.show()
    "☐ 정산 및 급여 관리",     # → page_settlement.show()
    "☐ 프로젝트 상세확인",    # → page_project_detail.show()
    "☐ AI 비서",              # → page_ai_assistant.show()
    "☐ 데이터 관리",          # → data_management.show_data_management()
]
```

페이지 모듈 (11개)

파일	메뉴명	주요 기능
<code>page_dashboard.py</code>	경영 대시보드	KPI 카드, D-Day 현황, 계약대기/미체결 목록, 수익 차트
<code>page_inquiry.py</code>	문의 접수	고객 문의 입력폼, 자동 ID 생성, 행사 일정 파싱
<code>page_estimate.py</code>	견적 통합 관리	품목별 견적, 복수안 비교, 견적서 이미지 생성·발송, 완료 프로젝트 조회
<code>page_contract.py</code>	계약 관리	카드형 목록, 계약 승인, 사업자등록증 OCR
<code>page_staff_new.py</code>	인원 배정 관리	3단계 배정 (후보→직군→확정), 팀배정, 출석/지급/평가 탭
<code>page_attendance.py</code>	출석부 관리	스케줄표, 일일 출근/퇴근 기록, 근무시간 계산
<code>page_settlement.py</code>	정산 및 급여	전체 정산 현황, 건별 정산, 세금계산서
<code>page_ceo.py</code>	대표님 전용	미수금/미지급, 수익보고, 인사 컨펌, 지급 현황
<code>page_project_detail.py</code>	프로젝트 상세	문의→정산 전체 흐름 조회, 고객 카드
<code>page_ai_assistant.py</code>	AI 비서	자연어 경영 조회, 기간별 리포트, 브리핑
<code>page_guide.py</code>	사용 가이드	단계별 업무 안내 (정적 콘텐츠)

데이터 계층

파일	역할
<code>data_loader.py</code>	핵심 데이터 계층. Google Sheets CRUD, 커넥션 풀링, 캐시 관리, 모든 시트 접근
<code>data_management.py</code>	데이터 초기화, 문의ID 기준 전체 삭제, 관리자 기능

비즈니스 로직

파일	역할
<code>calculators.py</code>	견적·급여·세금 자동 계산, 입력값 검증
<code>smart_assignment.py</code>	인력 스마트 매칭 (필터링 + 점수 산출 + AI 추천)
<code>workflow_automation.py</code>	배정→출석부→지급 자동 연계
<code>ai_helper.py</code>	매출 예측(이동평균), 리스크 분석, 고객 이탈 분석

유틸리티 (Brain 클래스 포함)

파일	핵심 클래스/함수	역할
<code>utils_estimate.py</code>	<code>EstimateBrain</code>	단가 조회, 직군 정보, 견적서 HTML/이미지 생성
<code>utils_staff.py</code>	<code>StaffBrain</code>	인력 점수 파싱, 추천도 평가, 출석부 HTML
<code>utils_settlement.py</code>	<code>SettlementBrain</code>	지급 현황 파싱, 거래명세서/급여명세서 생성
<code>utils_inquiry.py</code>	<code>InquiryParser</code>	이메일/전화 파싱, 날짜 스마트 변환
<code>utils_dashboard.py</code>	집계 함수들	KPI 계산, 곧 나갈 현장 (D-10), 미체결 목록
<code>utils_contract.py</code>	카드 HTML 생성	계약 카드 렌더링, 발송 상태 추적
<code>utils.py</code>	범용 유틸리티	안전한 숫자 변환, 시간 계산, Base64 인코딩
<code>helpers.py</code>	<code>now_kst()</code> , <code>@retry</code>	KST 시간, 로거, 재시도 데코레이터

외부 연동

파일	역할
<code>ocr_utils.py</code>	사업자등록증 OCR (Google Vision → EasyOCR → Pytesseract 폴백)
<code>scheduler.py</code>	APScheduler 일일(09:00, 17:00)/월간(1일 08:00) 자동 작업
<code>notifications.py</code>	이메일(SMTP)/Slack Webhook 알림 발송
<code>gemini_client.py</code>	Gemini API 래퍼. 민감정보 자동 마스킹 후 API 호출

설정 파일

파일	역할
<code>status_config.py</code>	상태 정의, 전환 규칙, UI 아이콘·색상 매핑
<code>project_context.py</code>	프로젝트 설계 문서 (코드 주석 형태)
<code>secrets.json</code>	GCP 서비스 계정 자격증명 (절대 Git 커밋 금지)

백업 파일

`_home_backup/` 디렉토리 및 `*.bak`, `*_backup.py` 파일들은 개발 중 생성된 백업이며, 프로덕션에서 사용하지 않습니다.

5. 데이터 계층 — Google Sheets 구조

스프레드시트 ID

13gzHX1p-oMznZjVfYR93RV1Zj5YAa10m56FWYU-p7RI

Google Sheets URL: https://docs.google.com/spreadsheets/d/{SHEET_ID}

트랜잭션 시트 (실시간 데이터)

시트명	코드 내 키	역할	주요 컬럼
문의작성	"inq"	고객 문의 원장	문의ID, 업체명, 행사명, 장소, 일시, 담당자, 연락처, 상태, 복장, 식사, 주차, 특이사항
견적상세	"estimate"	견적 메타	문의ID, 공급가액, 부가세, 합계금액, 발송방법, 발송메모
견적품목	"estimate_items"	견적 품목 상세	문의ID, 구분(인력/부대), 품목, 규격, 수량, 일수, 매출단가, 매입단가, 매출합계, 매입합계, 비고
견적안	"estimate_versions"	복수 견적안	문의ID, 견적안명, 품목데이터(JSON), 메타데이터(JSON), 생성일시
배정기록	"dispatch"	인력 배정 기록	배정ID, 문의ID, 행사명, 인력명, 직무, 팀코드, 구분(외부/본사), 지급단가, 근무일수, 총지급액, 상태
출석부	"attendance"	일일 출석 기록	기록ID, 배정ID, 문의ID, 인력명, 출석날짜, 출근시간, 퇴근시간, 근무시간, 출석상태
지급내역	"payment"	급여 지급 명세	지급ID, 배정ID, 문의ID, 인력명, 근무일수, 기본급, 교통비, 식비, 기타, 소계, 세금공제, 최종지급액, 지급상태, 별도정산, 비고
평가표	"evaluation"	인력 평가	평가ID, 배정ID, 인력명, 현장명, 근태, 수행, 외모, 팀워크, 총점, 평가등급
계약건은청 구금액적 기	"settlement"	정산 마스터	문의ID, 현장명, 업체, 파견일자, 청구금액, 공급가액, 부가세, 받은금액, 잔액, 진행상황, 입금여부, 지급액, 이익

참조 시트 (설정/마스터 데이터)

시트명	코드 내 키	역할	주요 컬럼
STAFF	"staff"	인력 데이터베이스	이름, 성별, 나이, 키, 영어, 운전면허, 거주지, 가능직무, 추천도, 근태/수행/외모/팀워크 점수, 총점, 비고
고객정보	"client"	거래처 DB	업체명, 대표자, 업태, 종목, 사업자번호, 사업자주소, 담당자, 연락처, 이메일
Roles	"roles"	직군 정의 + 기본 단가	직군명, 매출단가(기본), 매입단가(기본), 팀장추가비
Factors	"factors"	평가 항목 배점	항목명, 만점, 가중치, 등급기준
Guides	"guides"	업무 가이드 콘텐츠	단계, 제목, 내용, 주의사항

데이터 로딩 방식

```
# data_loader.py
data = db.get_data()
# 반환 구조:
{
    "df_inq": pd.DataFrame,      # 문의작성 시트
    "df_est": pd.DataFrame,      # 견적상세 시트
    "df_staff": pd.DataFrame,    # STAFF 시트
    "df_client": pd.DataFrame,   # 고객정보 시트
    "df_roles": pd.DataFrame,    # Roles 시트
    "df_factors": pd.DataFrame,  # Factors 시트
    "df_guides": pd.DataFrame,   # Guides 시트
}

dispatch_data = db.get_dispatch()
# 반환 구조:
{
    "df_dispatch": pd.DataFrame,  # 배정기록 시트
    "df_settlement": pd.DataFrame, # 계약건은청구금액적기 시트
    "df_payment": pd.DataFrame,   # 지급내역 시트
    "df_attendance": pd.DataFrame, # 출석부 시트 (일부 페이지)
}
```


6. 업무 파이프라인 (7단계 상태 흐름)

상태 정의

```
# status_config.py
STATUS_FLOW = [
    "접수",      # 0: 고객 문의 접수
    "견적",      # 1: 견적서 작성 완료
    "체결",      # 2: 계약 확정
    "배정완료",  # 3: 인력 배정 완료
    "진행중",    # 4: 행사 진행 중
    "완료",      # 5: 행사 종료
    "정산완료",  # 6: 급여/청구 정산 완료
]

STATUS_EXIT = ["미체결", "보류", "취소"] # 이탈 상태
```

상태 전환 규칙

```
STATUS_TRANSITIONS = {
    "접수":      ["견적", "미체결", "보류", "취소"],
    "견적":      ["체결", "미체결", "보류", "취소", "접수"],
    "체결":      ["배정완료", "보류", "취소"],
    "배정완료": ["진행중", "체결", "보류", "취소"],
    "진행중":    ["완료", "보류"],
    "완료":      ["정산완료"],
    "정산완료": [], # 최종 완료 (변경 불가)
    "미체결":    ["접수"], # 재활성화 가능
    "보류":      ["접수", "견적", "체결", "배정완료"],
    "취소":      [], # 최종 취소
}
```

단계별 상세 업무 흐름

Step 1: 문의 접수 (`page_inquiry.py`)

```
입력: 업체명, 행사명, 장소, 일시, 필요인원, 복장/식사/주차, 특이사항
|
▼ (자동) UUID[:8] 형식으로 문의ID 생성
|
▼ db.append_row("문의작성", row_data)
|
|_____ 상태: "접수", 대시보드 알림 카운트 +1
```

Step 2: 견적 산출 (`page_estimate.py`)

```

문의 선택 → 문의 데이터 자동 로드 (업체명, 행사일, 인원 등)
|
▼ 견적 품목 입력 (품목명, 수량, 일수, 매출단가, 매입단가)
|—— 매출합계 = 수량 × 일수 × 매출단가 (자동계산)
|—— 매입합계 = 수량 × 일수 × 매입단가
|—— 예상이익 = 매출합계 - 매입합계
|
▼ 부대비용 입력 (교통비, 식비, 장비비 등)
|
▼ 최종 금액 계산
|—— 공급가액 = 품목합계 + 부대비용합계 - 할인액
|—— 부가세 = 공급가액 × 0.1 (VAT 포함 선택 시)
|—— 합계금액 = 공급가액 + 부가세
|
▼ db.save_estimate_details() + db.save_estimate_items()
|
▼ 견적서 이미지 생성 (Pillow) → 카카오톡/이메일 발송
|
|—— 상태: "견적"

```

Step 3: 계약 승인 (`page_contract.py`)

```

카드형 목록에서 견적 건 선택
|
▼ 고객 정보 확인 (사업자등록증 OCR 옵션)
|
▼ 계약일자 입력 → 승인 버튼
|
▼ 정산 마스터("계약건은청구금액적기") 시트에 청구 정보 등록
|
|—— 상태: "체결"

```

Step 4: 인력 배정 (`page_staff_new.py`) — 3단계**Step 1**: 후보 등록

- └─ STAFF 시트에서 필터 검색 (성별, 나이, 추천도, 가능직무, 지역)
- └─ SmartAssignment.ai_recommend() → AI 매칭 점수 산출
- └─ 선택 인력 → 후보폴(assign_cart)에 저장
- └─ db.save_candidates_batch() → 배정기록 시트에 "후보" 상태로 저장

Step 2: 직군별 배정

- └─ 견적품목에서 직군 목록 자동 추출 (_get_role_status)
- └─ 각 직군별 필요인원 vs 배정인원 진행률 표시
- └─ 후보폴 → 직군 할당 (db.batch_assign_to_role)

Step 3: 확정 & 일정 관리

- └─ 인력별 근무 일정 입력 (일반: 단일 기간, 장기: 매트릭스)
- └─ [배정 확정] → db.batch_confirm_assignments()
- └─ 상태: "배정완료"

Step 5: 출석 관리 (`page_attendance.py`)

배정 확정된 건 선택

- |
- ▼ 스케줄표: 배정 인력 × 행사 일자 매트릭스
- |
- ▼ 각 인력별 출근시간/퇴근시간 입력
- |
- ▼ 근무시간 자동 계산 (퇴근 - 출근)
- |
- ▼ db.batch_save_attendance()
- |
- └─ 상태 수동 전환: "진행중" → "완료"

Step 6: 정산 (`page_settlement.py`)**TAB 1: 전체 현황**

- |—— 미입금/부분입금 건 목록
- |—— 입금여부 업데이트 (미입금/부분입금/입금완료)
- |—— 잔액 = 청구금액 - 받은금액 (자동계산)

TAB 2: 건별 정산

- |—— 인력별 지급액 입력 (근무일수 × 지급단가)
- |—— 세금공제 자동 계산
- |—— 지급기록 일괄 저장 (`db.batch_save_payment_records`)
- |—— 본사인원 → 자동 0원 확인완료 처리

TAB 3: 세금계산서

- |—— 발행 정보 입력 → 세금계산서 HTML/PDF 생성

Step 7: 최종 완료

- 정산 완료 확인 → `db.update_status(inquiry_id, "정산완료")`
- |
- |—— `page_project_detail`에서 전체 히스토리 조회 가능

7. 인증 및 GCP 설정

서비스 계정 자격증명 구조

```
{
  "type": "service_account",
  "project_id": "gradius-system",
  "private_key_id": "키 ID",
  "private_key": "-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n",
  "client_email": "python-bot@gradius-system.iam.gserviceaccount.com",
  "client_id": "숫자 ID",
  "auth_uri": "https://accounts.google.com/o/oauth2/auth",
  "token_uri": "https://oauth2.googleapis.com/token",
  "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
  "client_x509_cert_url": "..."
}
```

필요 API 스코프

```
SCOPES = [
    "https://www.googleapis.com/auth/spreadsheets", # Sheets 읽기/쓰기
    "https://www.googleapis.com/auth/drive",        # 파일 관리
    "https://www.googleapis.com/auth/calendar",      # 캘린더 (선택적)
]
```

인증 우선순위 (auth.py)

```
get_gspread_client()
1. secrets.json 파일 존재 시 → from_json_keyfile_name()
2. st.secrets['gcp_service_account'] 존재 시 → from_json_keyfile_dict()
3. st.secrets['service_account'] 존재 시 → from_json_keyfile_dict()
4. 모두 실패 → None 반환, 앱 중단
```

GCP 콘솔 설정 체크리스트

- ☐ Google Sheets API 활성화
- ☐ Google Drive API 활성화
- ☐ Google Calendar API 활성화 (캘린더 기능 사용 시)
- ☐ 서비스 계정 생성 (python-bot@gradius-system.iam.gserviceaccount.com)
- ☐ 서비스 계정에 JSON 키 발급 → secrets.json으로 저장
- ☐ Google Sheets 문서에 서비스 계정 이메일 공유 (편집자 권한)

8. 데이터 로더 API 레퍼런스

초기화

```
import data_loader as db
# data_loader는 모듈 레벨에서 초기화
# 페이지 모듈에서: import data_loader as db
```

데이터 읽기

```
# 메인 데이터 (문의, 견적, 인력, 고객, Roles, Guides)
data = db.get_data()
df_inq = data['df_inq']
df_est = data['df_est']
df_staff = data['df_staff']

# 배정/정산 데이터
dispatch = db.get_dispatch()
df_dispatch = dispatch['df_dispatch']
df_settlement = dispatch['df_settlement']
df_payment = dispatch['df_payment']
```

캐시 무효화

```
db.invalidate_data()          # 전체 캐시 (문의+배정 모두)
db.invalidate_main_only()     # 문의/견적 관련 변경 후
db.invalidate_dispatch_only() # 배정/정산 관련 변경 후
db.invalidate_payment_cache() # 지급내역 변경 후
```

규칙: 데이터를 쓴 후 반드시 해당 캐시를 무효화하고 `st.rerun()` 호출

상태 변경

```
db.update_status(inquiry_id, "견적")    # 상태 업데이트
db.update_cell("문의작성", inquiry_id, "담당자", "홍길동") # 특정 셀 업데이트
```

견적 저장

```
# 견적 메타데이터 저장
db.save_estimate_details(est_data_dict, metadata_dict)

# 견적 품목 저장 (인력 + 부대비용 한 번에)
db.save_estimate_items(inquiry_id, items_df, additional_costs_df)

# 견적안 버전 관리
db.save_estimate_version(inquiry_id, "견적안1", items_df, metadata)
versions = db.load_estimate_versions(inquiry_id)
```

배정 관련

```
# 후보 등록 (배치)
db.save_candidates_batch(inquiry_id, event_name, [
    {"이름": "홍길동", "직무": "경호원", "구분": "외부", ...},
])

# 직군 배정 (배치)
db.batch_assign_to_role([
    {"배정ID": "...", "직무": "경호원", "지급단가": 150000, "근무일수": 2},
])

# 배정 확정
db.batch_confirm_assignments(assign_ids_list, long_term=False)

# 일정 업데이트
db.batch_update_schedule([
    {"배정ID": "...", "날짜": "2026-04-15", "출근": "09:00", ...},
])
```

지급/정산

```
# 지급기록 배치 저장
db.batch_save_payment_records([
    {"문의ID": "...", "인력명": "홍길동", "근무일수": 2, "기본급": 300000, ...},
])

# 지급 상태 업데이트
db.batch_update_payment_status([
    {"assign_id": "...", "status": "확인완료", "pay_date": "2026-04-20"},
])

# 정산 마스터 업데이트
db.update_settlement_progress(inquiry_id, "정산완료")
```

9. 캐싱 전략

3-레벨 캐싱 아키텍처

```

Level 1: Google Sheets API (외부)
└── 요청마다 네트워크 I/O (느림, 매번 호출 X)

Level 2: @st.cache_data (프로세스 메모리)
├── load_all_data()      TTL=1800 (30분)
├── load_dispatch_data()  TTL=1800 (30분)
├── load_dispatch_sheet() TTL=120  (2분)
└── load_estimate_items() TTL=120  (2분)

Level 3: st.session_state (세션 메모리)
├── _app_data → get_data() 반환값 캐시
└── _dispatch_data → get_dispatch() 반환값 캐시

```

@st.cache_resource (싱글턴)

```

@st.cache_resource
def _get_cached_client():    # gspread.Client 싱글턴 (인증 1회)
    ...

@st.cache_resource
def _get_cached_spreadsheet(): # Spreadsheet 객체 싱글턴
    ...

```

캐시 무효화 흐름

```

# 데이터 변경 후 반드시 이 순서로:
db.save_something(...)    # 1. Sheets에 쓰기
db.invalidate_main_only()  # 2. 캐시 삭제 (session_state)
# st.cache_data는 자동 만료(TTL) 또는 _collect_all_cache 클리어
st.rerun()                # 3. 리렌더 (get_data() 다시 호출)

```

수동 동기화

앱 사이드바의 "데이터 동기화" 버튼:

```

db.invalidate_data()    # 전체 세션 캐시 삭제
st.cache_data.clear()   # @st.cache_data 전체 클리어
st.rerun()

```


10. 페이지 모듈 레퍼런스

pagestaffnew.py — 가장 복잡한 모듈

주요 함수 목록

함수	라인	설명
<code>show(data)</code>	-	메인 진입점 (4개 탭 라우팅)
<code>apply_styles()</code>	27	CSS 스타일 주입
<code>_col(df, *candidates)</code>	54	컬럼명 후보 중 존재하는 것 반환
<code>_select_contract(df_inq, statuses, key)</code>	80	계약 선택 드롭다운
<code>_get_role_status(est_items, assignments_df)</code>	142	직군별 배정 현황 계산
<code>_lookup_staff_brief(df_staff, name)</code>	240	이름으로 인력 기본정보 조회
<code>_auto_update_status(inquiry_id, role_status)</code>	282	배정 완료 시 상태 자동 전환
<code>_search_staff(df_staff, ...)</code>	315	다중 조건 인력 검색
<code>tab_assignment(data)</code>	401	배정 탭 전체 렌더링
<code>_step1_candidate_pool(...)</code>	569	Step1: 후보풀 UI
<code>_step2_role_assignment(...)</code>	1039	Step2: 직군배정 UI
<code>_step3_confirm_and_schedule(...)</code>	1620	Step3: 확정/일정 UI
<code>tab_attendance(data)</code>	1805	출석부 탭
<code>tab_payment(data)</code>	2022	지급현황 탭

getrole_status() 반환 구조

```
[
  {
    "role": "경호원",          # 직군명
    "needed": 5,              # 필요 인원
    "needed_mandays": 10,     # 필요 인일 (인원 × 일수)
    "assigned_count": 3,      # 현재 배정 인원
    "actual_mandays": 6,      # 현재 배정 인일
    "pay_rate": 150000,       # 지급단가
    "days": 2,               # 기본 일수
    "complete": False,        # 배정 완료 여부
    "has_date_items": False,   # 날짜별 품목 여부
    "date_details": {},        # 날짜별 인원 {날짜: 인원}
  },
  ...
]
```

page_estimate.py — 견적 관리

주요 함수 목록

함수	라인	설명
<code>show(data)</code>	165	메인 진입점
<code>_load_existing_items(inquiry_id)</code>	132	기존 견적 품목 로드
<code>_show_send_status_section(...)</code>	2124	견적 발송 현황 섹션
<code>_show_history_tab(...)</code>	2263	견적 버전 히스토리 탭
<code>_show_auto_recommend(...)</code>	2393	AI 단가 자동 추천

프로젝트 대기열 구분

```
"[접수] 업체명 (행사명)" → pending_new (STATUS_FLOW[0])
"[수정] 업체명 (행사명)" → pending_edit (STATUS_FLOW[1])
"[체결수정] 업체명 (행사명)" → pending_contracted (STATUS_FLOW[2~4])
"[완료] 업체명 (행사명)" → pending_completed (STATUS_FLOW[5~6])
```

page_settlement.py — 정산

주요 함수 목록

함수	라인	설명
<code>show(data)</code>	60	메인 진입점
<code>_auto_check_event_completion(df)</code>	77	행사 종료 자동 감지

함수	라인	설명
<code>show_settlement_overview()</code>	128	TAB1: 전체 현황
<code>show_settlement_detail(data)</code>	844	TAB2: 건별 정산
<code>update_payment_and_profit(...)</code>	766	청구/지급액 동기화
<code>show_tax_invoice_management()</code>	2621	TAB3: 세금계산서

11. 비즈니스 로직 모듈

calculators.py

```

from calculators import EstimateCalculator, SalaryCalculator, InvoiceCalculator

# 견적 계산
calc = EstimateCalculator()
supply = calc.calc_supply_price(items)          # 공급가액
supply, vat, total = calc.calc_total_with_tax(supply, vat_included=True)
profit, margin_pct = calc.calc_margin(supply, cost)

# 급여 계산
salary = SalaryCalculator()
result = salary.calc_staff_salary(assign_records) # {name: 급여액}

# 세금계산서 집계
invoice = InvoiceCalculator()
monthly = invoice.aggregate_monthly(contracts, 2026, 4)

```

smart_assignment.py

```
from smart_assignment import SmartAssignment, StaffFilter

# 필터링
sf = StaffFilter()
df_filtered = sf.apply_filters(df_staff, df_dispatch, {
    "gender": "F",
    "age_range": (25, 40),
    "skills": ["서빙", "안내"],
    "location": "서울",
    "availability": (start_date, end_date, "경호원"),
})

# AI 추천
engine = SmartAssignment()
top5 = engine.ai_recommend(
    staff_df=df_staff,
    dispatch_df=df_dispatch,
    job_type="경호원",
    location="서울",
    gender="M",
    age_range=(25, 40),
    start_date=start_d,
    end_date=end_d,
    top_n=5,
)

# 반환: [{"이름": ..., "score": 87.3, "이유": "..."}, ...]
```

HQ_STAFF 상수

```
# data_loader.py 내 정의 (L3255 근처)
HQ_STAFF = [
    {"이름": "최규성", "직무": "현장총괄", "구분": "본사"},
    {"이름": "송무재", "직무": "현장관리", "구분": "본사"},
    {"이름": "여지은", "직무": "현장관리", "구분": "본사"},
    {"이름": "김영찬", "직무": "현장관리", "구분": "본사"},
]

# 사용: from data_loader import HQ_STAFF
_hq_names = [m["이름"] for m in HQ_STAFF]

# 본사인원은 수익률 계산에서 제외, 정산 시 0원 자동확인완료
```

helpers.py

```
from helpers import now_kst, today_kst, get_logger, retry

# KST 현재 시각
now = now_kst()      # timezone-aware datetime

# KST 오늘 날짜
today = today_kst()  # naive datetime (비교용)

# 로거
logger = get_logger("page_estimate")
logger.info("견적 저장 완료")

# 재시도 데코레이터
@retry(times=3, delay=1, backoff=2)
def fetch_from_sheets():
    ...
```

12. AI 및 자동화

Gemini AI 연동

모델: `gemini-2.5-flash`

파일: `gemini_client.py`, `page_ai_assistant.py`, `ai_helper.py`

```
# gemini_client.py
from gemini_client import GeminiClient

client = GeminiClient()

# 민감정보 자동 마스킹 후 전송
response = client.ask(
    question="이번 달 매출은?",
    context_data={
        "settlements": df_settlement.to_dict(),
        "staff": df_staff[['이름', '직무']].to_dict(), # 민감컬럼 사전 제거
    }
)
```

민감정보 자동 마스킹 규칙:

- 주민번호 패턴 (`\d{6}-\d{7}`) → `[MASKED]`

- 계좌번호 패턴 → [MASKED]
- private_key, password 키 → 값 삭제

ai_helper.py 기능

```
from ai_helper import predict_monthly_revenue, analyze_risk

# 매출 예측 (3개월 이동평균)
forecast = predict_monthly_revenue(df_settlement, months_ahead=3)
# 반환: [{"month": "2026-05", "predicted": 15_000_000, "confidence": "높음"}]

# 리스크 분석
risks = analyze_risk(df_inq, df_settlement, df_dispatch)
# 반환: {"미수금_건수": N, "미배정_건수": N, "임박_현장": [...]}
```

스케줄러 (scheduler.py)

```
# APScheduler 기반 백그라운드 작업
from apscheduler.schedulers.background import BackgroundScheduler

scheduler = BackgroundScheduler()
scheduler.add_job(morning_check, 'cron', hour=9, minute=0) # 일일 오전 체크
scheduler.add_job(evening_check, 'cron', hour=17, minute=0) # 일일 오후 체크
scheduler.add_job(monthly_report, 'cron', day=1, hour=8) # 월간 리포트
```

workflow_automation.py

```
from workflow_automation import WorkflowEngine

engine = WorkflowEngine(db)

# 배정 확정 시 출석부/지급내역 자동 생성
engine.on_assignment_confirmed(inquiry_id, assignments)

# 행사 완료 시 정산 준비 알림
engine.on_event_completed(inquiry_id)
```

13. Session State 키 맵

전역 키

키	타입	설명
<code>_data_loaded_at</code>	str (HH:MM:SS)	마지막 데이터 동기화 시각
<code>_app_data</code>	dict	메인 데이터 캐시 (dfinq, dfest 등)
<code>_dispatch_data</code>	dict	배정/정산 데이터 캐시
<code>_inq_headers_checked</code>	bool	문의 시트 헤더 자동 확장 실행 여부
<code>_nav_target</code>	str	사이드바 네비게이션 타겟

견적 페이지 키 (w_ 접두사 = Workspace)

키	타입	설명
<code>w_client</code>	str	업체명
<code>w_event</code>	str	행사명
<code>w_loc</code>	str	장소
<code>w_manager</code>	str	담당자
<code>w_contact</code>	str	연락처
<code>w_sdate</code>	date	행사 시작일
<code>w_edate</code>	date	행사 종료일
<code>w_date_periods</code>	list[(date,date)]	다중 기간
<code>w_date_text</code>	str	날짜 직접입력 텍스트
<code>w_qty</code>	int	필요 인원
<code>w_dress</code>	str	복장
<code>w_meal</code>	str	식사
<code>w_parking</code>	str	주차
<code>w_note</code>	str	특이사항
<code>est_items</code>	DataFrame	견적 품목 테이블
<code>additional_costs</code>	DataFrame	부대비용 테이블
<code>vat_yn</code>	bool	VAT 포함 여부

키	타입	설명
<code>discount_amt</code>	int	할인액
<code>est_project_sel</code>	str	선택된 프로젝트 라벨
<code>last_project</code>	str	이전 선택 프로젝트 (중복 로드 방지)
<code>_current_inq_id</code>	str	현재 문의ID
<code>dp_s_N</code> / <code>dp_e_N</code>	date	N번째 기간 시작/종료
<code>_tab2_gen</code>	int	탭 세대 카운터 (위젯 키 리셋용)

배정 페이지 키

키	타입	설명
<code>assign_cart</code>	list	후보폴 입력 목록
<code>search_done</code>	bool	검색 완료 여부
<code>search_results</code>	DataFrame	검색 결과
<code>_editing_assign_id</code>	str	수정 중인 배정ID
<code>matrix_days_RI_CI</code>	int	매트릭스 셀별 일수
<code>matrix_dates_RI_CI</code>	list	매트릭스 셀별 날짜

14. 개발 환경 설정

1. 레포지토리 클론

```
git clone https://github.com/ghgh2472-glitch/gradius_python.git
cd gradius_python
```

2. Python 환경 설정

```
python -m venv venv
source venv/bin/activate      # macOS/Linux
# venv\Scripts\activate      # Windows

pip install -r requirements.txt
```


3. 시스템 패키지 (Ubuntu/Debian)

```
sudo apt-get install fonts-nanum # 한글 폰트 (견적서 이미지 생성 필수)
```

4. 자격증명 설정

```
# GCP 서비스 계정 JSON을 secrets.json으로 저장
cp /path/to/your-service-account.json secrets.json
```

⚠ 주의: `secrets.json` 은 절대 Git에 커밋하면 안 됩니다. `.gitignore` 에 이미 포함되어 있습니다.

5. 앱 실행

```
streamlit run app.py --server.enableCORS false --server.enableXsrfProtection false
```

브라우저에서 `http://localhost:8501` 접속

6. 개발 시 유용한 명령어

```
# 특정 포트 지정
streamlit run app.py --server.port 8502

# 디버그 모드 (파일 변경 시 자동 리로드 비활성화)
streamlit run app.py --server.runOnSave false

# 캐시 클리어 후 재시작
streamlit cache clear && streamlit run app.py
```

15. 배포 가이드

Streamlit Community Cloud 기준

`.streamlit/secrets.toml`

```
[gcp_service_account]
type = "service_account"
project_id = "gradius-system"
private_key_id = "..."
private_key = "-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n"
client_email = "python-bot@gradius-system.iam.gserviceaccount.com"
client_id = "..."
auth_uri = "https://accounts.google.com/o/oauth2/auth"
token_uri = "https://oauth2.googleapis.com/token"
auth_provider_x509_cert_url = "https://www.googleapis.com/oauth2/v1/certs"
client_x509_cert_url = "..."
```

`.streamlit/config.toml`

```
[server]
maxUploadSize = 50          # MB, 사업자등록증 업로드용
enableCORS = false
enableXsrfProtection = false

[theme]
primaryColor = "#2563EB"
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#F8FAFC"
textColor = "#1E293B"
font = "sans serif"
```

자체 서버 (Docker) 기준

```
FROM python:3.11-slim

RUN apt-get update && apt-get install -y fonts-nanum
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .

EXPOSE 8501
CMD ["streamlit", "run", "app.py", \
    "--server.enableCORS=false", \
    "--server.enableXsrfProtection=false", \
    "--server.port=8501"]
```

배포 전 체크리스트

- secrets.json → 환경 변수 또는 시크릿 매니저로 이전
- requirements.txt 최신 상태 확인
- packages.txt에 fonts-nanum 포함 확인
- Google Sheets API Rate Limit 확인 (초당 100 요청)
- 서비스 계정이 스프레드시트 편집자로 공유되어 있는지 확인
- Gemini API 키 시크릿에 추가 (GEMINI_API_KEY)

16. 주요 설계 패턴

1. 상태 머신 패턴 (Status Machine)

모든 프로젝트는 **STATUS_FLOW**를 따르며, **STATUS_TRANSITIONS**에 정의된 전환만 허용됩니다.

```
# status_config.py 활용 예시
import status_config as sc

# 현재 상태에서 이동 가능한 다음 상태 목록
next_options = sc.get_next_statuses(current_status)

# 상태 배지 HTML 생성
badge_html = sc.get_status_badge_html("체결")

# 진행률 바
progress = sc.get_status_progress("배정완료") # 50 (%)
```

2. Brain 클래스 패턴

도메인별 복잡한 비즈니스 로직을 클래스로 캡슐화합니다.

```
# 새 Brain 클래스 작성 예시
class NewFeatureBrain:
    def __init__(self, df_data1, df_data2):
        self.data1 = df_data1
        self.data2 = df_data2

    def calculate_something(self, inquiry_id: str) -> dict:
        # 로직 구현
        ...
        return result
```

3. 컬럼명 유연 조회 패턴

Google Sheets 컬럼명이 버전에 따라 다를 수 있으므로, 후보 목록에서 존재하는 컬럼을 찾는 패턴을 사용합니다.

```
# page_staff_new.py의 _col() 패턴
def _col(df, *candidates):
    for c in candidates:
        if c in df.columns:
            return c
    return candidates[0] # 없으면 첫 번째 반환 (KeyError 대신 명시적 실패)

# 사용 예시
name_col = _col(df, '이름', '인력명', '성명')
role_col = _col(df, '직무', '직군', '역할')
```

4. 세션 상태 백업/복원 패턴

Streamlit 탭 전환 시 위젯 상태가 리셋되는 문제를 방지합니다.

```
# 저장 전 백업
if 'w_client' in st.session_state:
    st.session_state['_bak_w_client'] = st.session_state['w_client']

# 복원 시 체크
if '_bak_w_client' in st.session_state and 'w_client' not in st.session_state:
    st.session_state['w_client'] = st.session_state.pop('_bak_w_client')
```

5. 배치 쓰기 패턴 (API 호출 최소화)

```
# 개별 호출 (나쁜 예 - API 호출 N번)
for record in records:
    worksheet.append_row(record) # N번 호출

# 배치 호출 (좋은 예 - API 호출 1번)
from gspread.cell import Cell

cells = []
for i, record in enumerate(records):
    row_num = existing_rows + i + 1
    for j, val in enumerate(record):
        cells.append(Cell(row=row_num, col=j+1, value=val))

worksheet.update_cells(cells, value_input_option='RAW') # 1번 호출
```

6. 탭 세대 카운터 패턴

프로젝트 전환 시 위젯 키 충돌을 방지합니다.

```
# 프로젝트 전환 시
st.session_state['_tab2_gen'] = st.session_state.get('_tab2_gen', 0) + 1

# 위젯 키에 세대 포함
gen = st.session_state.get('_tab2_gen', 0)
st.text_input("업체명", key=f"final_client_{gen}")
```

17. 알려진 기술 부채

항목	현황	개선 방안	우선순 위
동명이인 구 분	이름만으로 인력 식별 (성별/연락처 미 사용)	배정 시 복합키(이름+연락처) 저장, 조회 시 복 합 매칭	높음
데이터 정규 화	일부 데이터가 JSON 문자열로 셀에 저장됨	별도 컬럼으로 스키마 분리	중간
OCR 정확도	Pytesseract 한글 인식률 ~70%	Google Vision API 활성화 또는 GPT-4V 연동	낮음
동시성	Google Sheets API 동시 쓰기 시 충 돌 가능	낙관적 잠금(Optimistic Locking) 또는 큐 기 반 쓰기	중간
권한 관리	모든 사용자가 동일 접근 권한	역할 기반 접근 제어(RBAC) 구현	낮음

항목	현황	개선 방안	우선순 위
백업	Google Sheets에만 의존	주기적 Cloud Storage 내보내기	중간
테스트	자동화 테스트 부재	핵심 비즈니스 로직 단위 테스트 추가	중간

18. 개발 시 주의사항

절대 금지 사항

- ❌ secrets.json을 Git에 커밋하지 말 것
- ❌ Google Sheets를 직접 편집하지 말 것 (app 통해서만)
- ❌ st.cache_data로 데코레이팅된 함수 내에서 Sheets 쓰기 금지
- ❌ session_state 키를 중복 정의하지 말 것 (다른 페이지와 충돌)

새 기능 개발 가이드라인

1. 새 페이지 추가 시

```
# app.py에 메뉴 항목 추가
_menu_items.append("NEW 새 기능")

# 새 파일 생성: page_newfeature.py
def show(data):
    db = data.get('db') # or import data_loader as db
    df_inq = data.get('df_inq', pd.DataFrame())
    ...
```

2. 새 시트 컬럼 추가 시

```
# data_loader.py의 ensure_*_headers() 패턴 사용
def ensure_new_headers():
    ws = _get_worksheet("문의작성")
    headers = ws.row_values(1)
    new_cols = ["새컬럼1", "새컬럼2"]
    for col in new_cols:
        if col not in headers:
            ws.add_cols(1)
            ws.update_cell(1, len(headers)+1, col)
```

3. 데이터 쓰기 후 반드시

```
db.save_something(...)
db.invalidate_main_only() # 또는 invalidate_dispatch_only()
st.rerun()
```

4. 새로운 상태 추가 시

```
# status_config.py 수정 필요:
# - STATUS_FLOW 또는 STATUS_EXIT에 추가
# - STATUS_CONFIG에 icon/color/bg/desc 추가
# - STATUS_TRANSITIONS에 전환 규칙 추가
# - ACTIVE_STATUSES, CONFIRMED_STATUSES 등 그룹 상수 업데이트
```

성능 고려사항

- Google Sheets API **100 요청/초** Rate Limit 존재
- 대량 데이터 쓰기 시 반드시 `batch_*` 메서드 사용
- 자주 읽는 데이터는 `@st.cache_data` 로 캐싱
- `st.rerun()` 남발 금지 (매 호출마다 전체 스크립트 재실행)

코드 스타일

- 한국어 변수명 허용 (컬럼명, UI 레이블)
- 영어 함수명 사용 (스네이크 케이스)
- 내부 헬퍼 함수는 `_` 접두사 (`_get_role_status`, `_lookup_staff_brief`)
- 페이지별 CSS는 `apply_styles()` 함수에 집중

부록: 자주 쓰는 코드 스니펫

특정 문의ID의 전체 데이터 조회

```
inq_id = "ABC12345"
row = df_inq[df_inq['문의ID'].astype(str).str.strip() == inq_id]
est = df_est[df_est['문의ID'].astype(str).str.strip() == inq_id]
dispatch_rows = df_dispatch[df_dispatch['문의ID'].astype(str).str.strip() == inq_id]
```

상태 뱃지 표시

```
import status_config as sc
status = "체결"
st.markdown(sc.get_status_badge_html(status), unsafe_allow_html=True)
```

안전한 숫자 변환

```
from utils import safe_int, safe_float
amount = safe_int(row.get('금액', 0)) # None/NaN/' ' → 0
rate = safe_float(row.get('수익률', 0))
```

KST 현재 시각 기록

```
from helpers import now_kst
timestamp = now_kst().strftime("%Y-%m-%d %H:%M:%S")
```

페이지 내 네비게이션 이동

```
# 다른 페이지로 이동 (app.py의 _nav_map 활용)
st.session_state['_nav_target'] = "정산"
st.rerun()
```

이 문서는 Gradius ERP master 브랜치 기준으로 작성되었습니다. 코드 변경 시 문서도 함께 업데이트해 주세요.